



Lab Title: Security Misconfiguration

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS103 – Owasp Top 10 Vulnerabilities
Exploitation Techniques

Instructor Name: AMMAR AMINU

Date of Submission: 01/02/2026

Executive Summary

In this lab, I examined how poor security settings in a web application can expose sensitive information. I worked with a deliberately misconfigured PHP application where important files such as config.php and .htaccess were accessible through the browser. By directly accessing these files, I was able to view confidential details like database credentials. After identifying the issue, I fixed the misconfiguration by restricting access through the .htaccess file and moving sensitive files outside the public web directory. This lab showed me how small configuration mistakes can lead to serious security problems and how proper setup can prevent them.

Lab Objective

The objective of this lab was to help me understand security misconfiguration in a simple and practical way. I aimed to identify insecure settings in a web application, see how an attacker could exploit them to access sensitive files, and then apply proper configurations to secure the application. By the end of the lab, I focused on learning how to protect important files and reduce security risks through correct setup and file management.

Lab Environment

I performed this lab on a local web server running on my machine. The environment included a PHP-based web application hosted on localhost, using an Apache web server. I created basic files such as index.php, .htaccess, and config.php to simulate a real web application. This setup allowed me to safely test the vulnerability, exploit it, and apply fixes without affecting a real production system.

Task 1: Exploiting Security Misconfiguration

Step 1: Setting up the Vulnerable PHP Application

- I created new folder [lab9_SecurityMisconfiguration](#).

```

(raph@kali)-[/opt/lampp/htdocs]
$ sudo mkdir lab9_SecurityMisconfiguration
[sudo] password for raph:

(raph@kali)-[/opt/lampp/htdocs]
$ ls
applications.html
bitnami.css
dashboard
favicon.ico
img
index.php
lab10_xss
lab17_Insecure_Deserialization
Lab2_HTMLInjection
Lab4_CommandInjection
lab8_BrokenAccessControl
lab9_SecurityMisconfiguration
test.php
vulnerable_app
webalizer
week2_php_lab

(raph@kali)-[/opt/lampp/htdocs]
$ cd lab9_SecurityMisconfiguration

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$

```

- Inside this folder, I created three files called :
 - **index.php** (a basic page of the web application)
 - **.htaccess** (an insecure .htaccess file)
 - **config.php** (a configuration file with sensitive information)

```

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ sudo nano index.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ sudo nano .htaccess

```

```

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfi
guration]
$ sudo nano config.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfi
guration]
$ ls
config.php  index.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfi
guration]
$ ls -a
.  ..  config.php  .htaccess  .htaccessy  index.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfi
guration]
$ █

```

Step 2: Implemented the Vulnerable Code

- **Index.php:** This is a simple web page that stimulate a login screen and displays some sensitive information.

```

Session  Actions  Edit  View  Help
GNU nano 8.6      index.php *
<?php
// Check if the user is logged in
session_start();

if (!isset($_SESSION['username'])) {
    echo "<p>Please log in to view the page.</p>";
} else {
    echo "<h2>Welcome to the User Dashboard</h2>";
    echo "<p>This page is for logged-in users.</p>";
}
?>

```

- **.htaccess:** If this file is not set up correctly, it can expose sensitive files or allow unauthorized users to access restricted parts of the application. This happens because the file is publicly accessible through the web, which is a basic security misconfiguration.

```
Session Actions Edit View Help
GNU nano 8.6 .htaccess *
# .htaccess with improper configurations
# This file should restrict access to sensitive files, >

<Files ".ht*">
  Order Allow,Deny
  Deny from all
</Files>

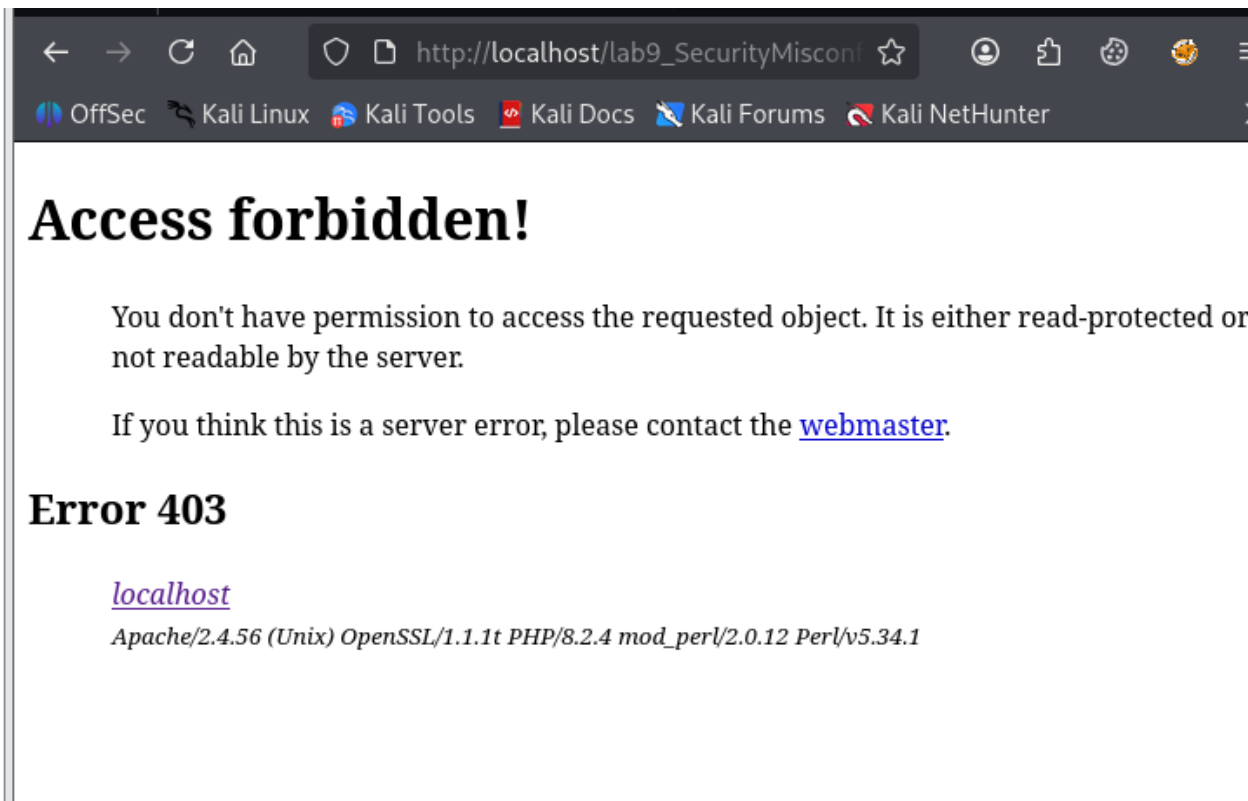
# But it's missing the configuration to block access to >
```

- **config.php**: This file stores sensitive information like database login details. If the system is not properly configured, attackers can access this file and view the confidential data.

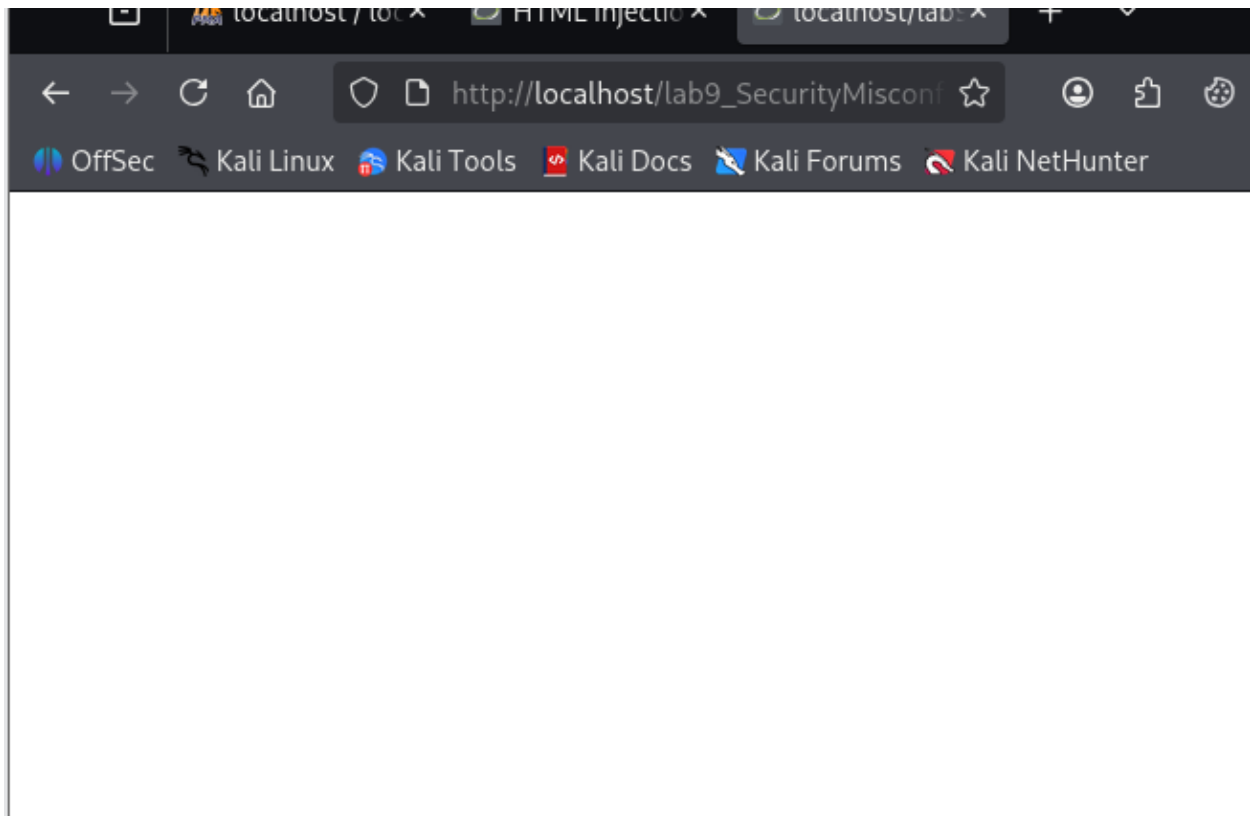
```
raph@kali: /opt/lampp/htdocs/lab9_SecurityMisconfiguration
Session Actions Edit View Help
GNU nano 8.6 config.php *
<?php
// Database credentials - this file should NOT be public
$db_host = 'localhost';
$db_user = 'root';
$db_pass = 'password';
$db_name = 'mydatabase';
```

Step 2: Exploiting the vulnerability

1. When I tried to access the .htaccess file through the browser, the server returned a **403 Forbidden** error. This showed that access to the file was correctly restricted and the file was not publicly accessible.



2. I tried to access the config.php file directly through the browser by visiting `http://localhost/config.php`, but the page was blank. This happened because the file only contains variable definitions and no output statements. If the file were not properly secured and contained visible output, it could display sensitive information such as database credentials.



Reflection Questions:

- What kind of sensitive information can attackers gain by exploiting misconfigurations?

Attackers can obtain database usernames and passwords, system paths, configuration settings, and other private details that can help them take control of the application or access stored data.

- How can access to `.htaccess` or `config.php` be restricted to prevent unauthorized access?

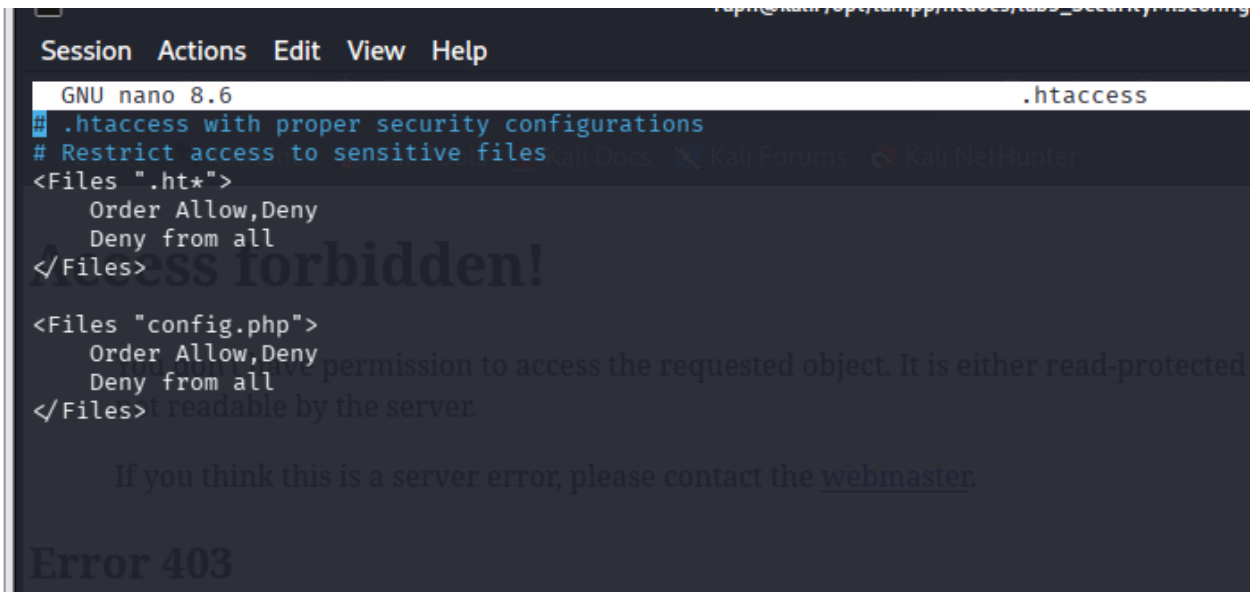
Access can be restricted by properly configuring the `.htaccess` file to block sensitive files, moving `config.php` outside the public web folder, and setting correct file permissions so only the server can read the files.

Task 2: Mitigate Security Misconfiguration

Step 1: Correctly Configure `.htaccess` .

To fix this misconfiguration, access to sensitive files such as `.htaccess` and `config.php` must be blocked. This can be done by properly securing the `.htaccess` file as shown below.

- Updated the `.htaccess` file to restrict access to sensitive files:



```
Session Actions Edit View Help
GNU nano 8.6 .htaccess
# .htaccess with proper security configurations
# Restrict access to sensitive files
<Files ".ht*">
    Order Allow,Deny
    Deny from all
</Files>

<Files "config.php">
    Order Allow,Deny
    Deny from all
</Files>

If you think this is a server error, please contact the webmaster

Error 403
```

Step 2: Secure the `config.php` file

```

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ mkdir bvwa
mkdir: cannot create directory 'bvwa': Permission denied

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ sudo mkdir bvwa

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ ls
bvwa  config.php  index.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ sudo mv config.php bvwa

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ ls
bvwa  index.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration]
$ cd bvwa

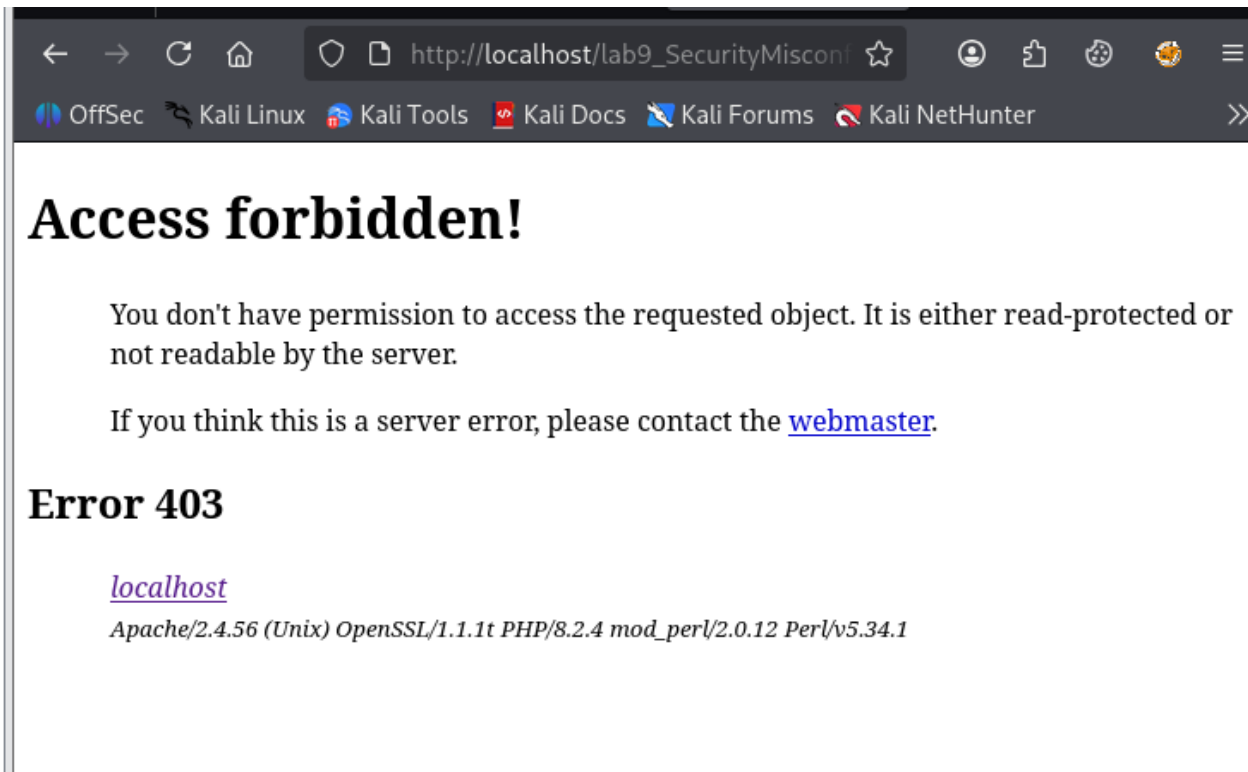
(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration/bvwa]
$ ls
config.php

(raph@kali)-[/opt/lampp/htdocs/lab9_SecurityMisconfiguration/bvwa]
$

```

mouse pointer inside of press Ctrl + G

Step 3: Test the Mitigated Version



Reflection Questions:

- How did moving the config.php file out of the public web root improve security?

By taking the config.php file out of the public folder, it means people on the internet cannot directly open it through a browser. This file usually contains sensitive details like database username and password. If it stayed in the public web root, hackers could try to access it. Moving it outside makes it harder for outsiders to see or steal those details.

- How do proper file permissions and access control contribute to the overall security of the web application?

Setting the right permissions on files means only the right people or processes can read, write, or run them. For example, the web server might be allowed to read a file but not change it. This reduces the chance of someone editing or deleting important files. Access control also ensures that users only get to see what they are supposed to, which helps keep the application safe from misuse or attacks.

Key Takeaway

This lab showed me that poor configuration settings can expose sensitive files and put a web application at risk. Even simple mistakes, like leaving configuration files accessible, can lead to serious security problems.

Conclusion

In conclusion, I learned that security misconfiguration is a common but preventable issue. By properly configuring `.htaccess`, protecting sensitive files, and using correct file permissions, I can reduce the risk of unauthorized access and build more secure web applications in the future.